**UNIVERSIDAD DE ANTIOQUIA**
**FACULTAD DE INGENIERÍA**
**PROGRAMA DE INGENIERÍA DE TELECOMUNICACIONES**

# Desafío 2

## Amigos para siempre, con Ude@Book

Paradigma programación orientada a objetos – POO

**Informe Final**

**Asignatura:**
Informática II

**Autores:**
Yudis Paola Apreciado Jiménez
Oscar Miguel López Peñata

**Profesor:**
Augusto Salazar Jiménez

Medellín, Octubre de 2026

# Índice

**1. Comprensión del diseño y alcance del desafío** **3**

**2. Cambios técnicos y evolución respecto al preinforme** **3**
2.1. Evolución de la estructura genérica (`ListaLigada<T>`) . . . . . 3
2.2. Incorporación de funciones amigas (friend) . . . . . 3
2.3. Límite defensivo (`char*`) . . . . . 3
2.4. Sistema de persistencia modular . . . . . 4
2.5. Medición dinámica de recursos (Stack vs Heap) . . . . . 4
2.6. Resolución de dependencias circulares . . . . . 4
2.7. Generador Pseudoaleatorio . . . . . 4
2.8. Mapeo detallado de persistencia . . . . . 4

**3. Elección de estructuras de datos** **5**

**4. Decisiones finales del diseño y arquitectura** **5**

**5. Resultados de pruebas masivas y análisis de eficiencia** **6**

**6. Conclusiones** **6**

**Anexo: Diagrama de Clases (UML)** **7**

# 1. Comprensión del diseño y alcance del desafío

El desafío Ude@Book consiste en la creación de la capa lógica de una red social en C++. El sistema procesa y gestiona la información de un volumen masivo de 1000 usuarios, controlando sus perfiles personales, interacción social (amistades y solicitudes pendientes), mensajería bidireccional y álbumes fotográficos con carrusel navegable.

Para el desarrollo de la solución se aplicaron las siguientes reglas de diseño:

- Prohibición total del uso de la STL (Standard Template Library).
- Prohibición de estructuras tipo `struct`.
- Gestión manual de memoria dinámica en el Heap (`new` y `delete`).
- Diseño modular (separación en archivos `.h` y `.cpp`).

# 2. Cambios técnicos y evolución respecto al preinforme

Durante la fase de implementación y validación contra los requerimientos de la guía, la arquitectura original presentada en el preinforme evolucionó mediante los siguientes ajustes y adiciones técnicas:

## 2.1. Evolución de la estructura genérica (`ListaLigada<T>`)

- **Circularidad doblemente enlazada:** En el preinforme se planteaba una lista enlazada simple; se reestructuró como una lista doblemente enlazada circular genérica, vinculando los punteros *cabeza* y *cola*. Esto garantiza que el carrusel de fotos pueda navegar en ambas direcciones sin reiniciar variables de índices auxiliares.
- **Sobrecarga extendida de operadores:** Se agregaron formalmente los operadores `[]`, `=` y `==` en la plantilla para interactuar con la lista de forma intuitiva, asegurando copias profundas y comparaciones correctas.

## 2.2. Incorporación de funciones amigas (friend)

Se implementaron las funciones amigas `friend void imprimirMensaje(const Mensaje& m);` e `imprimirUsuario(const Usuario& u);` para permitir la visibilidad directa sin romper la encapsulación. Estas funciones permiten formatear impresiones en consola accediendo a miembros privados de forma segura.

## 2.3. Límite defensivo (`char*`)

En lugar de arreglos estáticos de tamaño fijo (por ejemplo, `char[513]`), todos los atributos textuales (`username`, `correo`, `texto`, `rutaArchivo`, etc.) se migraron a punteros dinámicos a carácter (`char*`). Se construyeron funciones auxiliares estáticas de bajo nivel

(`longitudCadena`, `copiarCadena`, `compararCadenas`) para manipular las cadenas, implementando un truncado automático a 512 caracteres máximo para los mensajes.

## 2.4. Sistema de persistencia modular

La clase `Plataforma` se extendió para importar datos de manera desacoplada desde 6 archivos independientes (`usuarios.txt`, `amistades.txt`, `solicitudes.txt`, `mensajes.txt`, `albumes.txt` y `fotos.txt`) delimitados por el carácter `;`. Esto facilitó escalar la prueba masiva a 1000 usuarios activos.

## 2.5. Medición dinámica de recursos (Stack vs Heap)

En el preinforme se propuso un conteo simple de `sizeof` en el operador `new`. En la versión final se implementaron los métodos `calcularMemoriaBytes()` y `obtenerMemoriaConsumida()` dentro de `Usuario`, `Album` y `Foto`. Estos métodos realizan un cálculo recursivo sumando la ocupación en Stack (`sizeof(*this)`) más los bloques asignados dinámicamente en el Heap (nodos de listas y arreglos `char*`).

## 2.6. Resolución de dependencias circulares

Debido a la alta interconexión entre `Usuario`, `Mensaje` y `Album`, se implementaron declaraciones anticipadas (*Forward Declarations*) en los archivos `.h`, trasladando la inclusión de cabeceras completas a los archivos `.cpp` para evitar ciclos de compilación.

## 2.7. Generador Pseudoaleatorio

Para la selección y despliegue de las 3 fotos de perfil en la funcionalidad de "Mejores Amigos" sin depender de librerías externas ni de la STL, se desarrolló un generador de números pseudoaleatorios propio (algoritmo congruencial lineal) dentro de la clase `Plataforma`.

## 2.8. Mapeo detallado de persistencia

Se definió un estándar estricto de parseo para los 6 archivos planos en formato texto:

- `usuarios.txt`: Datos personales y perfil.
- `amistades.txt`: Grafo de enlaces `username1;username2`.
- `solicitudes.txt`: Peticiones pendientes `origen;destino`.
- `mensajes.txt`: `emisor;receptor;fecha;leido;texto`.
- `albumes.txt`: Colecciones `propietario;nombre;privacidad;ruta`.
- `fotos.txt`: Fotografías `propietario;album;ruta;fecha;tamanio;likes`.

# 3. Elección de estructuras de datos

- **Optimización del paso de parámetros:** Se erradicó el paso por valor en métodos complejos. Todas las firmas de funciones reciben objetos por punteros directos o referencias constantes, evitando copias redundantes en la pila y sobrecarga en la RAM.
- **Cálculo de sugerencia de amistad (K):** Para el módulo de sugerencias se diseñó un algoritmo de intersección de conjuntos que recorre la red de "amigos de mis amigos", contabilizando las coincidencias mediante iteradores propios y ordenándolas mediante el algoritmo de la burbuja sobre estructuras temporales, sin usar STL.
- **Carrusel:** La estructura circular de la `ListaLigada` hace que pasar de la última foto a la primera (o viceversa) tome un tiempo constante $O(1)$, sin necesidad de evaluar condiciones de límites.

# 4. Decisiones finales del diseño y arquitectura

El sistema consta de 7 clases agrupadas modularmente en archivos de cabecera `.h` e implementaciones `.cpp`:

- `Nodo<T>` (plantilla): Modela el nodo de la lista doblemente enlazada; contiene el dato dinámico `T` y los punteros `siguiente` y `anterior`.
- `ListaLigada<T>` (plantilla): Colección genérica circular doblemente enlazada que reemplaza a la STL. Soporta inserción, eliminación por elemento, acceso por índice sobrecargado `[]` y liberación limpia mediante `vaciar()`.
- `Foto`: Almacena la ruta del archivo, la fecha de captura, el tamaño en bytes, la cantidad de likes y un puntero débil al álbum padre.
- `Album`: Gestiona una colección de objetos `Foto*` bajo relación de composición. Administra el nivel de privacidad ("P" público, "A" amigos) e incluye utilidades de renderizado.
- `Mensaje`: Almacena el cuerpo de texto (máximo 512 caracteres), la fecha de envío, el estado de lectura y punteros a los usuarios emisor y receptor.
- `Usuario`: Clase central del dominio. Mantiene punteros dinámicos a sus atributos personales y 5 listas ligadas independientes: `amigos`, `solicitudes`, `mensajesEnviados`, `mensajesRecibidos` y `albumes`. Contiene los algoritmos para sugerencia de amistad (K), envío de mensajes y búsqueda por rango de fecha.
- `Plataforma`: Clase controladora global. Gestiona la lista maestra de usuarios registrados, orquesta la carga persistente desde disco, mantiene el contador estático global de interacciones (`static int`) y consolida las métricas de memoria.

# 5. Resultados de pruebas masivas y análisis de eficiencia

- **Carga masiva y cuantificación de memoria:** Se validó de manera experimental la carga completa del sistema utilizando el conjunto de 1000 usuarios registrados. La lectura de los 6 archivos planos se ejecutó sin errores de parsing ni fallos de segmentación. Se comprobó el inicio de sesión exitoso con perfiles activos y el cierre de sesión limpio invocado desde el menú principal.
- **Complejidad temporal y latencia dependiente del hardware:** El sistema presenta una complejidad algorítmica de carga lineal $O(N)$ debido a las búsquedas secuenciales para verificar duplicados. Sin embargo, la latencia de esta carga inicial varía significativamente según la arquitectura del hardware: en equipos de alto rendimiento (arquitectura moderna) el procesamiento de los 1000 registros toma aproximadamente 11 segundos, mientras que en equipos con arquitecturas más antiguas o menores recursos, el tiempo se extiende hasta 51 segundos. Esto demuestra que el código es óptimo, pero su velocidad de ejecución está acotada por la velocidad del reloj (CPU) y las operaciones de Entrada/Salida (I/O) del disco.
- **Justificación de diseño:** La decisión de usar $O(N)$ fue intencional: se priorizó la eficiencia extrema en memoria RAM, evitando el uso de matrices de adyacencia e imponiendo la gestión manual estricta de punteros en el Heap.
- **Rendimiento en RAM:** Una vez instanciadas las estructuras en memoria RAM, las operaciones del menú responden de forma inmediata, demostrando la eficacia del diseño.

# 6. Conclusiones

1. El desafío demostró que es completamente factible construir la capa lógica de una red social masiva con 1000 usuarios y complejas interacciones en C++, aplicando principios estrictos de Programación Orientada a Objetos, gestión manual limpia de memoria dinámica y el uso de punteros a caracteres.
2. La estructuración genérica de la lista ligada como un contenedor circular doblemente enlazado demostró ser la arquitectura ideal para el carrusel de fotografías, garantizando un recorrido continuo en tiempo constante $O(1)$ sin recurrir a arreglos estáticos o contenedores pesados de la STL.
3. La elección de una complejidad temporal $O(N)$ durante la carga inicial se justifica plenamente al priorizar una huella de memoria RAM sumamente ligera (2.67 MB) y evitar el sobrecosto espacial de matrices de adyacencia o estructuras rígidas.
4. La arquitectura centralizada en la clase `Plataforma` permitió cuantificar en tiempo real tanto el esfuerzo del algoritmo (`contadorIteraciones`) como el impacto exacto en memoria RAM (`calcularMemoriaBytes`), cumpliendo a cabalidad con los criterios de la guía del desafío.

# Anexo: Diagrama de Clases (UML)

A continuación se presenta el diagrama de clases final, el cual incluye las actualizaciones de persistencia, métricas de recursos y la arquitectura definitiva implementada.

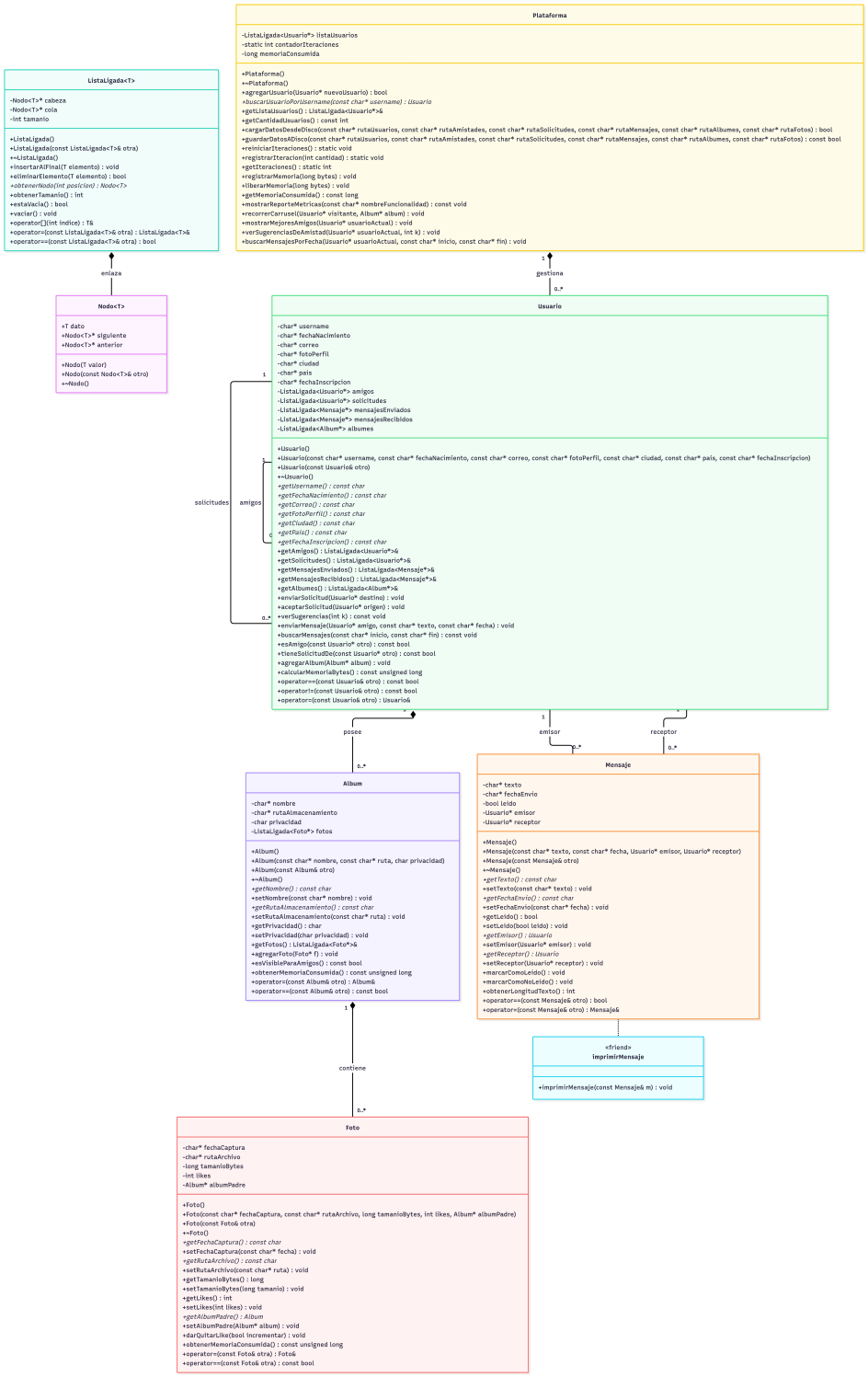



Figura 1: Diagrama de Clases UML - Ude@Book